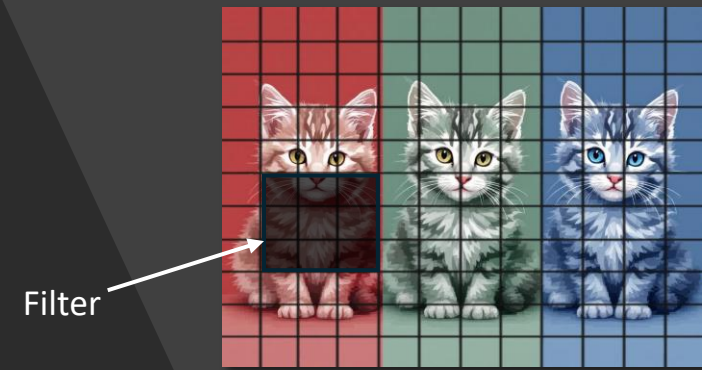


CNNs – Fusion Opportunity



Naive Execution Problems:

R: Read from Memory

W: Write to Memory

Input $\rightarrow$ R Conv2D W $\rightarrow$ R BatchNorm W $\rightarrow$ R ReLU W $\rightarrow$ Output

Each operation writes intermediate results to memory

- Separate kernel launches for each operation
- Poor data locality
- Underutilized hardware resources

Optimization Opportunity:

Input $\rightarrow$ R [Conv2D + BatchNorm + ReLU] W $\rightarrow$ Output

- Fused operations eliminate intermediate memory writes and reduce kernel launches.
- It is like taking a direct highway instead of stopping at every traffic light!

Kernel Fusion Techniques

Merging Operations for Efficiency

Common Fusion Patterns:

1. Conv-BatchNorm-Activation

- $conv_output = conv2d(input, weights) + bias$
- $norm_output = batch_norm(conv_output, scale, offset)$
- $final_output = relu(norm_output)$
- Each step writes to memory and reads from memory

After fusion, the above operations become a single (fused) kernel with no intermediate memory writes.

2. Element-wise Operation Fusion

Unfused:

Fused

- $result[i] = relu((x[i] + y[i]) \times z[i])$

Compiler-based Graph Optimization

Automated Transformation Systems

Pattern Recognition and Rewrite Rules:

```
# Compiler identifies patterns and applies transformations
pattern = "Conv2D → BatchNorm → ReLU"
replacement = "FusedConvBnRelu"

# Systematic application across entire graph
for subgraph in computation_graph:
    if matches_pattern(subgraph, pattern):
        replace_with(subgraph, replacement)
```

Multi-stage Optimization:

1. **High-level transformations:** Operation fusion, redundancy elimination
2. **Hardware-specific optimizations:** Layout transformations, kernel selection
3. **Low-level optimizations:** Instruction scheduling, register allocation

Optimization Verification:

Ensure mathematical equivalence through formal verification or numerical testing.

